

StampFly ファームウェアの現状報告

— アーキテクチャ・開発経緯・工夫点 —

第 64 回飛行機シンポジウム原稿準備資料（内部報告）

Kouhei Ito

2026 年 8 月 1 日

目次

1	本報告の目的と位置づけ	2
2	機体とファームウェアの全体像	3
2.1	ハードウェア諸元	3
2.2	ソフトウェア層構造（6 層）	3
2.3	4 階層アクセス制御（学習者向け並列 API、横断ルール R8）	4
2.4	Pub-Sub トピック通信	4
2.5	BSP 層（sf_board）の責務	5
2.6	タスク構成と周期	5
2.7	飛行モードと状態遷移	5
2.8	アーキテクチャ不変条件（INV）	6
2.9	コーディング／教育方針の要点	6
3	位置制御系の構成	7
3.1	4 段カスケード構成	7
3.2	PID 形式（不完全微分・D-on-M・条件付き積分）	7
3.3	加速度→傾き写像 $a \approx g\theta$	7
3.4	既定パラメータ値（params.cpp）	8
3.5	高度制御（ALT_HOLD）	9
3.6	加速度 DOB（外乱オブザーバ）	9
3.7	ホバー推力学習	10
3.8	スティック・リポジショニング機能	10
3.9	自動離陸シーケンス（ALT/POS_HOLD）	10
4	状態推定（ESKF）	11
4.1	状態数と内訳	11
4.2	観測	11
4.3	フロー→速度変換パイプライン	11
4.4	ロバスト化 5 機構	12
4.5	α - β トラッカ（運動加速度補償）	12
4.6	アクチュエータ遅れ（ ≈ 110 ms）の内訳	12

5	開発の経緯（時系列）	13
5.1	POS_HOLD「実効ゲイン欠損」の詳細	14
6	工夫点の整理（論文の材料候補）	15
6.1	(a) 飛行ログからの実効ゲイン欠損の定量同定	15
6.2	(b) 点校正でなく変動域へのロバスト再設計	15
6.3	(c) SIL（実ファームのホスト実行・決定論）と Model Identity 還流	15
6.4	(d) 鉛直 DOB（IMU 比力直接使用で ESKF 遅れ 28ms を回避する設計）	15
6.5	(e) オンボード・ホバー推力学習（NVS 永続化）	16
6.6	(f) 実装細部（条件付き積分 AW・フェーズ別 ti・D-on-M 等）	16
6.7	(g) INV による構造規律	16
6.8	(h) 教育目的の 4 階層アクセス	16
7	実飛行実績の数値一覧	16
8	文書・実装・記録の食い違い（生の報告）	17
9	原稿執筆に向けた現状	18
9.1	MANUSCRIPT_PLAN.md §5 残作業チェック（facts_history.md §3 照合）	18
9.2	解析レポート・スクリプト棚卸し	18
9.3	締切までの残日数	19
10	未確認事項一覧	19

1 本報告の目的と位置づけ

本報告は、第 64 回飛行機シンポジウム講演「StampFly の位置制御に関する一考察」（締切 2026-08-07）の原稿方針を決めるための、事実整理を目的とした内部資料である。論文そのものではない。

原稿の構成計画は 2026-07-02 に承認され、docs/hikoki64/MANUSCRIPT_PLAN.md として記録済みである（コミット 2180f0dc）。本報告作成時点（2026-08-01）で締切まで 6 日である。

本報告の出典は以下の 4 件の調査資料に限定する。いずれもリポジトリ stampfly_ecosystem 現物（ソースコード・設計文書・git 履歴・解析レポート）を出典としたサブエージェント調査の原文、および主担当による直接検証の追補である。

- facts_architecture.md — ファームウェア全体アーキテクチャ
- facts_implementation.md — POS_HOLD/状態推定の実装詳細
- facts_history.md — 開発経緯（git 履歴）と原稿準備状況
- facts_verification_addendum.md — 高度 DOB 既定値の食い違いに関する直接検証

方針として、材料にない事実は創作しない。数値・日付・ファイルパスは材料の記載をそのまま転記する。材料が「未確認」とする項目は本報告でも「未確認」と明記する。文書・実装・記録の間の食い違いを発見した場合も隠さず記録する（第 8 章）。誇張・宣伝的な表現は用いず、淡々とした技術報告として記述する。

項目	値	出典
MCU	ESP32-S3 (レイヤード構成図の最下層に明記)	firmware/vehicle/docs/architecture.md:36, 645
機体質量	0.037kg (kMassG = 0.037f * 9.80665f [N]、コメントに「機体質量」と明記)	firmware/vehicle/components/sf_controller_pid/include/pid_controller.hpp:407, 468--470

2 機体とファームウェアの全体像

2.1 ハードウェア諸元

指定 4 文書 (requirements.md / architecture.md / hardware_init.md / coding_and_education.md) には機体質量の数値記載はない。上表の質量値はコンポーネントのソースコード (sf_controller_pid) から確認した値であり、設計文書には未記載である (第 8 章で再掲)。

■センサー一覧 (requirements.md §5 「搭載センサ」表、requirements.md:135-147)

センサ	型番	通信/レート	用途
IMU	BMI270	SPI / 400 Hz	姿勢推定の主センサ
地磁気	BMM150	I2C / 25 Hz	ヨー推定 (研究対象)
気圧	BMP280	I2C / 50 Hz	高度推定 (ToF 範囲外)
ToF (下面)	VL53L3CX	I2C / 30 Hz	高度計測
ToF (前面)	VL53L3CX	I2C / 30 Hz	障害物検知・SLAM・ナビゲーション
OpticalFlow	PMW3901	SPI / 100 Hz	水平位置推定
電源モニタ	INA3221	I2C / 10 Hz	バッテリー電圧・電流

備考: 「全センサ残す (IoT 教材として全アクセス可能であることが重要)」 (requirements.md:147)。前面・下面の ToF は同一型番 VL53L3CX で、コンポーネントとしては単一の sf_hal_vl53l3cx を Bottom/Front 用に 2 回インスタンス化する構成 (hardware_init.md:154, 158)。

■アクチュエーター一覧 (requirements.md §6, requirements.md:149-156)

アクチュエータ	制御方式	数量
モーター	LEDC PWM 150kHz 8bit	4 (X-quad 配置)
ブザー	LEDC PWM トーン生成	1
LED (MCU 内蔵)	WS2812 RGB	1
LED (ボード上下)	WS2812 RGB デイジーチェーン	2

2.2 ソフトウェア層構造 (6 層)

Application Layer (タスク: StateTask 等) → Service Layer (状態管理・通知・ログ・通信) → Algorithm Layer (状態推定・制御・フィルタ) → HAL Layer (IMU・Motor・LED・ToF 等) → BSP Layer (sf_board、共有 HW 資源の唯一の所有者: I2C/SPI バス・LEDC タイマー・esp_netif・event_loop・NVS) → Hardware (ESP32-S3 peripherals)。出典: architecture.md:22-38。

2.3 4 階層アクセス制御（学習者向け並列 API、横断ルール R8）

層	名前空間 / 公開ヘッダ	典型ユーザー	できること
L0: Sketch API	<code>ws::* / ws_api.hpp</code>	Workshop 受講者・初心者	<code>setup()/loop_400Hz(dt)</code> 、 <code>ws::motor_set_duty()</code> 等 30+ 関数で完結。HW・タスク・Topic 知識ゼロで飛行制御を体験
L1: Topic API	<code>sf::api::* / sf_api.hpp</code>	推定・制御・ガイダンス学習者	Topic を subscribe/publish し自作 ESKF/PID/Navigator を実装。 <code>IEstimator/IController</code> を実装して差替え
L2: HAL Direct	<code>stampfly::*Wrapper / 各 HAL の*_wrapper.hpp</code>	HW 学習者	<code>BMI270Wrapper.readSensorData()</code> 等を直接呼び、 SPI/I2C/RMT/LEDC を理解。 Topic を介さない経路
L3: BSP Internal	<code>sf::internal::board / sf_board.hpp</code> 等	ファーム実装者・拡張者	<code>sf_board</code> の getter で bus handle 取得、esp-idf 直叩き。 起動順序や HW 資源管理を変更可能

各層は並列に共存し、学習者は必要な層までしか降りない（`architecture.md:147-179`、`hardware_init.md:281-311`）。これは横断ルール R8「公開 API は `sf::api` namespace、内部 API は `sf::internal` namespace に分離」（`architecture.md:93`）の具体化である。

2.4 Pub-Sub トピック通信

同一 MCU 内タスク間通信のみを対象とし（プロセス間通信不要）、シリアルライズ不要（構造体メモリ直接共有）、全トピックはコンパイル時確定（`architecture.md:187-191`）。内部実装は用途別に使い分ける（`architecture.md:193-201`）：

- IMU →推定: Lock-free SPSC Ring Buffer（ISR 安全・400Hz ロスなし）
- ToF/Flow/Mag/Baro →推定: FreeRTOS Queue
- 推定値→制御: 共有メモリ + Task Notification（最新値のみ・最低レイテンシ）
- 全データ→ログ: Lock-free Ring Buffer（全サンプル保持）
- 推定値/モード→テレメトリ: 共有メモリ最新値（50Hz 間引き）

外部インターフェースは `topic.publish(data) / topic.subscribe(callback)` または `topic.latest()` に統一（`architecture.md:205-215`）。

トピック	内容	レート
<code>sensor.imu</code>	IMU 生データ	400 Hz
<code>sensor.tof</code>	ToF 距離	30 Hz
<code>sensor.flow</code>	オプティカルフロー	100 Hz
<code>sensor.mag</code>	地磁気	25 Hz
<code>sensor.baro</code>	気圧	50 Hz
<code>sensor.power</code>	電源	10 Hz
<code>estimate.state</code>	推定状態	400 Hz
<code>system.mode</code>	モード	イベント
<code>command.setpoint</code>	操縦目標	50 Hz
<code>control.output</code>	制御出力	400 Hz
<code>actuator.motor</code>	モータ指令	400 Hz
<code>system.alert</code>	アラート	イベント

出典: `architecture.md:219-232`。横断ルール R5「データ共有は Pub-Sub Topic のみ。複数 producer/複数 consumer が同一バッファに無ロック並走することを禁止」、R7「同期手段は 3 種類のみ: Pub-Sub Topic / `xTaskNotify/std::atomic::volatile bool` の散在を禁止」(`architecture.md:85, 87`)。

2.5 BSP 層 (sf_board) の責務

共有 HW 資源の唯一の所有者。出典: `hardware_init.md:69-77 / architecture.md:565-572`。

資源	ESP-IDF 型	借用する HAL/Service
I2C master bus	<code>i2c_master_bus_handle_t</code>	<code>sf_hal_bmp280</code> , <code>sf_hal_bmm150</code> , <code>sf_hal_vl53l3cx</code> , <code>sf_hal_power</code>
SPI host (IMU)	<code>spi_host_device_t</code>	<code>sf_hal_bmi270</code>
SPI host (Flow)	<code>spi_host_device_t</code>	<code>sf_hal_pmw3901</code>
LEDC timer	<code>ledc_timer_t</code>	<code>sf_hal_motor</code> , <code>sf_hal_led</code> , <code>sf_hal_buzzer</code>
<code>esp_netif</code> (STA) + event loop	<code>esp_netif_t</code> *等	<code>sf_comm</code> (WiFi/ESP-NOW) , <code>sf_telemetry</code> (UDP socket)
NVS default partition	<code>nvs handle</code>	<code>sf_calibration</code> , <code>params system</code>

各 HAL は `sf_board` から Config 経由で bus handle を借用する (extern グローバル禁止、R1~R2)。公開 API は `sf::internal::board::*` で、L3 ユーザー専用 (`hardware_init.md:63-101`)。

2.6 タスク構成と周期

出典: `architecture.md §6 「タスク設計」 (architecture.md:506-521、日英同一値)`。

タスク	周期	優先度	含むコンポーネント
<code>ImuTask</code>	400 Hz	24	センシング (IMU) + 状態推定
<code>ControlTask</code>	400 Hz(IMU 同期)	23	制御 + アクチュエーション
<code>StateTask</code>	イベント駆動	22	状態管理
<code>FlowTask</code>	100 Hz	20	センシング (OptFlow)
<code>MagTask</code>	25 Hz	18	センシング (Mag)
<code>BaroTask</code>	50 Hz	16	センシング (Baro)
<code>CommTask</code>	50 Hz	15	通信 + コマンド処理
<code>TofTask</code>	30 Hz	14	センシング (ToF) + 離着陸マネージャー
<code>TelemetryTask</code>	50 Hz	13	テレメトリ
<code>PowerTask</code>	10 Hz	12	センシング (Power) + フェイルセーフ
<code>ButtonTask</code>	50 Hz	10	ボタン入力
<code>NotifyTask</code>	30 Hz	8	通知 (LED/ブザー)
<code>CLITask</code>	20 Hz	5	CLI + パラメータ
<code>LogTask</code>	非同期	5	データロガー + Blackbox

タスク非割当のコンポーネント: キャリブレーション管理 (`StateTask` の `onEnter` コールバックから呼ばれ常駐不要)、ナビゲーター (将来独立タスク予定、初期実装は構造のみ)、BSP `sf_board` (`app_main()` Phase 1 で `sf::board::init()` を 1 回呼ぶのみ)。出典: `architecture.md:541-547`。

統合理由の例 (`architecture.md:525-531`): IMU+ 状態推定は「400Hz で密結合、タスク切替コスト削減」、制御 + アクチュエーションは「制御出力を即モーター反映、安全チェックも同タスク」。

コア (CPU0/CPU1) 割り当てについては、4 指定文書中に `xTaskCreatePinnedToCore` やコア番号の明記は未確認である (grep で該当記述なし)。別の auto-memory 記録に「真因=コア 1 飽和」という記述があり実装上コアピン留めがある可能性は高いが、指定文書には記載がなく確認できない。

2.7 飛行モードと状態遷移

出典: `requirements.md §2 「状態モデル」 (requirements.md:23-45)`。

INIT (逐次実行、サブモードなし)

→ IDLE (GROUND: ToF 無効距離・ARM 可・キャリブ可 / HELD: ToF 有効距離・ARM 禁止・推定テスト可)

- ARMED_GROUND (ARM、GROUND からのみ)
- TAKEOFF (シーケンス)
- FLYING (ACRO: 角速度制御 / STABILIZE: 姿勢安定 / ALT_HOLD: 高度維持 / POS_HOLD: 位置維持)
- LANDING (シーケンス)
- IDLE_GROUND

設計原則 (requirements.md:47-53) : ARM/DISARM はアクション (モードではない)、親状態が共通処理を保証、TAKEOFF/LANDING は専用モード (離着陸マネージャーが統括)、FAILSAFE は状態ではなくイベント。

離陸トリガはモードで分岐する。ALT_HOLD/POS_HOLD は ARM 自体がトリガ (スロットル入力不要、0.3s 整定後に自動離陸で目標高度 0.5m)、ACRO/STABILIZE は手動スロットル入力 (生スロットル=推力、自動離陸なし)。着陸も同様に、ALT_HOLD/POS_HOLD でのパイロット DISARM は緩降下の自動着陸、ACRO/STABILIZE でのパイロット DISARM は手動推力モードゆえ自動着陸を持たず即カット (requirements.md:62, 65, 68)。サブモード切替は地上 (IDLE_GROUND/ARMED_GROUND) と FLYING でのみ受理される (設置時の変更が最も安全なため、requirements.md:64)。

2.8 アーキテクチャ不変条件 (INV)

出典: architecture.md:108-119 (§2「横断ルール」直後の節)。

#	一行要約
INV-1	全鉛直フェーズ (Grounded/TakeoffClimb/Airborne/Landing) は単一の姿勢+レートパイプラインを共有し、独自の並列制御経路を持つてはならない。フェーズが変えてよいのは鉛直チャンネルのみ
INV-2	空中の全フェーズでパイロットの姿勢操縦 (roll/pitch/yaw) を奪わない。自動化は鉛直・水平位置等の並進軸に限定し、例外は「リンク途絶」判定のみ
INV-3	「検出」と「判断」を分離する。接地・離陸等の検出ロジックは検出層 (sf_takeoff_landing 等) に置き、StateManager が判断する
INV-4	状態機械の各 (状態 × 入力) セルは detailed_design.md§3.1 の規範表で全て規定され、表にない暗黙の振る舞いを作らない

INV 策定の背景: 「姿勢は常にパイロット」原則を確立したのに着陸 (computeLanding) が旧前提「フェイルセーフ専用・スティック無視」のまま並列経路として残っていたことによる実機バグ (着陸中ロール/ピッチが効かない) が契機 (2026-06-14、architecture.md:112)。

2.9 コーディング／教育方針の要点

出典: coding_and_education.md。

1. **模範的ソースコードという最重要目標**: 「vehicle のコードは単なるファームウェアではなく、ドローンのファームウェアを作ろうとする人が参考にできる模範的なソースコードであること、また学習教材として機能すること」 (coding_and_education.md:10)。
2. **可読性ルール** (coding_and_education.md:22-31) : 1 関数 1 責務・50 行以内、バイリンガルコメント (英語→日本語の順で全関数・全ブロック)、関数冒頭ドキュメント (3~5 行)、マジックナンバー禁止 (全数値に config 定数名/パラメータ名)、略語禁止、ネスト 2 段まで、Pub-Sub によるコンポーネント間直接呼び出し禁止。
3. **@design タグによる設計トレーサビリティ**: クラス定義・インターフェース実装・状態遷移コールバックに設計文書該当箇所を参照し判定ステータス ([OK]/[NG]/[--]) を付記。「リリース時点で全ての @design タグが [OK] であること」 (coding_and_education.md:34-46)。
4. **設計矛盾の即時報告**: 実装中に設計文書との矛盾を発見したら「実装を進めずに立ち止まる」「矛盾の内容を具体的に報告する」「議論して設計を更新する」「変更履歴を残す」の 4 段階 (coding_and_education.md:101-108)。
5. **4 段階の Examples/教育体系**: Level 1「ハードウェア基礎 (ESP-IDF とセンサ) — Tier L2 中心」、Level 2「通信と制御の基礎 — Tier L1/L2 混在」、Level 3「フライトシステム — Tier L1 中心」、Level 4「応用・拡張 — Tier L1 (差し替え) +L4 (応用)」に加え、Workshop Lesson が Tier L0 に対応 (coding_and_education.md:239-315、見出しのみ確認、本文詳細は未読)。

3 位置制御系の構成

3.1 4 段カスケード構成

実装は `firmware/vehicle/components/sf_controller_pid/pid_controller.cpp` の `compute()` 内で、以下の順にブロックが実行される。全て単一の `compute()` 呼び出し内で処理され、外側/内側ループ間の分周（サブサンプリング）は行われていない（`grep` で分周パターン不検出）。

ループ	入力	出力	制御器	出典
位置（外）	<code>pos_setpoint</code> <code>state.position</code>	$v_{x,sp}, v_{y,sp}$ [NED, m/s]	P (<code>pos_x_.compute</code> , <code>ti</code> はパラメータ上存在するが実質 P の運用)	<code>pid_controller.cpp</code> : 1326-1327
速度	<code>v_sp - state.velocity</code>	$a_{x, ned}, a_{y, ned}$ [NED, m/s ²]	PID	<code>pid_controller.cpp</code> : 1332-1333
（加速度→傾き写像）	$a_{x, body}, a_{y, body}$ (ヨー一回転後)	$roll_{sp}, pitch_{sp}$ [rad]、 $\pm max_pos_tilt_$ でクランプ	$\theta = \mp a_{body}/g$	<code>pid_controller.cpp</code> : 1337-1351
姿勢	$roll_{sp}/pitch_{sp}$ <code>euler.x/y</code>	$rate_{sp, roll}/pitch$ [rad/s]	PID (<code>att_roll_, att_pitch_</code>)	<code>pid_controller.cpp</code> : 455-456
レート（内）	<code>rate_sp - gyro_rate</code>	<code>output.torque[0..2]</code> [Nm]	PID (D-on-M)	<code>pid_controller.cpp</code> : 774-776

高度系は並列チェーン: `alt_pos_.compute()`（高度→鉛直速度目標）→ `alt_vel_.compute()`（鉛直速度→推力補正）→ `thrust = hover_thrust_ + thrust_correction` (`pid_controller.cpp`:628-638)。

制御周期はヘッダコメントに「ControlTask は IMU 同期 400Hz で動く」と明記 (`pid_controller.hpp`:443-448, `kDobRateHz = 400.0f`, `pid_controller.hpp`:453)。位置ループの実効帯域が遅いのはゲイン設計によるものであり、サンプリング周期の分離ではない。関連文書に「位置ループ ≈ 0.06 Hz、速度ループ実効クロスオーバー ≈ 0.19 Hz」の記述があり、これもゲインに由来する (`poshold_wobble_velocity_noise.md`:65)。

3.2 PID 形式（不完全微分・D-on-M・条件付き積分）

実装は `firmware/vehicle/components/sf_controller_pid/include/pid.hpp` (struct PID)。伝達関数はファイルヘッダに明記される (`pid.hpp`:14)：

$$C(s) = K_p \left(1 + \frac{1}{T_i s} + \frac{T_d s}{\eta T_d s + 1} \right)$$

ここで K_p は比例ゲイン、 T_i は積分時間 [s]、 T_d は微分時間 [s]、 η は不完全微分係数 (既定 $\eta = 0.125$, `pid.hpp`:54) である。

- 微分は「測定値」に作用する (D-on-M)。理由は目標値ステップでの微分キック回避 (`pid.hpp`:20-31)。
- 双一次変換 (Tustin) で離散化: $\alpha = 2\eta T_d/dt$, $a = (\alpha - 1)/(\alpha + 1)$, $b = 2T_d/((\alpha + 1)dt)$ (`pid.hpp`:92-102, dt は制御周期 [s])。
- 積分は台形則+条件付き積分アンチwindアップ（出力が飽和方向に押される場合は積分更新を止める。ハードクランプはバックストップとして併用, `pid.hpp`:108-133)。
- $T_i \geq 0.01$ s の場合のみ積分実行（境界含む。過去バグの修正済み, `pid.hpp`:123)。

3.3 加速度→傾き写像 $a \approx g\theta$

`computePositionHold()` (`pid_controller.cpp`:1260-1352) 内、NED 加速度指令をヨーで機体座標へ回転後:

```
pitch_sp = clampTilt(-ax_body / gravity_);
roll_sp = clampTilt( ay_body / gravity_);
```

(pid_controller.cpp:1350-1351)。gravity_ = math::kGravity = 9.80665 m/s² (sf_math.hpp:39、pid_controller.hpp:223)。clampTilt は ±max_pos_tilt_ (既定 0.1745 rad = 10°、pid_controller.hpp:224) でクランプする。

3.4 既定パラメータ値 (params.cpp)

パラメータ	既定値	範囲	出典
position.pos.kp	0.4	[0, 10]	params.cpp:771
position.pos.ti	5.0 s	[0.1, 100]	params.cpp:772
position.vel.kp	3.0	[0, 10]	params.cpp:773
position.vel.ti	2.0 s	[0.1, 100]	params.cpp:774
position.stick_vel	0.4 m/s	[0.05, 2.0]	params.cpp:775

■位置・速度系 (POS_HOLD) X/Y 軸は同一ゲイン (pos_y_ = pos_x_; vel_y_ = vel_x_;、pid_controller.cpp:161-162)。

パラメータ	既定値	範囲	出典
attitude.roll.kp / attitude.pitch.kp	5.0	[0, 50]	params.cpp:719,722
attitude.roll.ti / attitude.pitch.ti	2.0 s	[0.01, 100]	params.cpp:720,723
attitude.roll.td / attitude.pitch.td	0.04 s	[0, 1]	params.cpp:721,724
rate.roll.kp	1.0×10^{-3}	[0, 0.01]	params.cpp:666
rate.roll.ti	0.7 s	[0.01, 100]	params.cpp:667
rate.roll.td	0.002 s	[0, 1]	params.cpp:668
rate.pitch.kp	1.426432×10^{-3}	[0, 0.01]	params.cpp:669
rate.pitch.td	0.025 s	[0, 1]	params.cpp:671
rate.yaw.kp	1.198594×10^{-3}	[0, 0.01]	params.cpp:672
rate.yaw.ti	0.8 s	[0.01, 100]	params.cpp:673
rate.yaw.max_torque	1.83×10^{-3} Nm	[1e-4, 2.1e-3]	params.cpp:677 (NT 金沢ヨー飽和治療)

■姿勢・レート系 (代表値) レートゲインの単位は「B⁻¹ ミキサー用の物理ゲイン [Nm/(rad/s)]」(params.cpp:662-663 コメント)。

■各段のリミット値 (pid_controller.hpp、コード内定数・param SSOT 外)

リミット	値	単位	出典 / 備考
max_roll_pitch_torque_	5.2×10^{-3}	Nm	pid_controller.hpp:590 (旧 ROLL/PITCH_OUTPUT_LIMIT)
max_att_rate_sp_	3.0	rad/s	pid_controller.hpp:168 (ACRO スティックスケール 1.0 とは別物)
max_pos_tilt_	0.1745 (=10°)	rad	pid_controller.hpp:224 (POS_HOLD 傾き上限)
max_pos_vel_	1.0	m/s	pid_controller.hpp:320 (POS_HOLD 速度目標上限)
max_thrust_correction_	0.15	N	pid_controller.hpp:200 (旧 VEL_OUTPUT_MAX)
max_thrust_	0.672	N	pid_controller.hpp:184 (4×0.168N/モータ、T/W≈1.85)
max_angle_	0.5236 (=30°)	rad	pid_controller.hpp:159 (STABILIZE 最大傾き)
max_rate_	1.0	rad/s	pid_controller.hpp:157 (ACRO スティック→レート指令スケール)
max_yaw_rate_	5.0	rad/s	pid_controller.hpp:158

出力上限の代入箇所は pid_controller.cpp:173-183。特に vel_x_.output_limit = gravity_ × max_pos_tilt_ ≈ 1.71 m/s² で、速度ループ出力が傾き上限と直結する (pid_controller.cpp:182-183)。

これらリミット値は params::get_float で読み込まれておらず (loadParams() 内に対応呼び出しなし)、コンパイル時定数としてハードコードされ、param SSOT (NVS 経由のライブ変更) の対象外である。

3.5 高度制御 (ALT_HOLD)

構成: altitude.alt.* (P) → altitude.vel.* (PI) → 推力補正。

パラメータ	既定値	出典
altitude.alt.kp	0.45	params.cpp:738
altitude.alt.ti	7.0 s	params.cpp:739
altitude.vel.kp	0.1	params.cpp:740
altitude.vel.ti (climb 用)	2.5 s	params.cpp:741
altitude.vel.ti_hover	2.5 s	params.cpp:742
altitude.climb_rate	0.5 m/s	params.cpp:743
altitude.descent_rate	0.5 m/s	params.cpp:744

鉛直速度ループの積分時間はフェーズ別スケジュールになっている (applyAltVelTiForPhase(), pid_controller.cpp:208-212): Airborne (ホバー) のみ alt_vel_ti_hover_、TakeoffClimb/Landing/Grounded は alt_vel_ti_climb_ を用いる。

3.6 加速度 DOB (外乱オブザーバ)

altitude.dob.fc が 0 以下 (既定 0.0f, params.cpp:745) なら完全無効。実装は computeDobCorrection() (pid_controller.cpp:1109-)。

- ・ノミナル・アクチュエーションモデル: kDobModelDelayS = 0.0624 s (純遅れ L 、飛行実測)、kDobModelLagS = 0.02 s (1 次遅れ T 、モータ時定数) (pid_controller.hpp:440-441)。
- ・Q フィルタ: 2 次バターワース LPF ($Q = 1/\sqrt{2}$ 、pid_controller.cpp:1038-1048)、カットオフは param altitude.dob.fc。
- ・ウォッシュアウト (1 次 HP): 固定 kDobWashoutHz = 0.03 Hz (pid_controller.hpp:438)、DC 所有権を速度ループ積分器とホバー推力学習に残す設計 (pid_controller.cpp:1089-1092)。
- ・動作周期: kDobRateHz = 400.0 (pid_controller.hpp:453)。

■既定昇格→同日差し戻しの経緯 (facts_verification_addendum.md による確定事実)

- ・2026-07-18 06:22 コミット ffbdae55 feat(control): promote altitude DOB to compiled default (fc=1.5) — DOB を既定有効化 (altitude.dob.fc を 1.5 へ)。
- ・2026-07-18 08:03 コミット 0a3d8a39 fix(control): revert altitude defaults to pre-DOB behavior (pilot urgent) — 同日約 100 分後にパイロット運用判断で差し戻し。altitude.dob.fc 1.5 → 0.0 (DOB 無効)、altitude.vel.ti_hover 1.5 → 2.5。dob.fc=0 で DOB コード経路は不活性 (SIL バイト同一を実装時に検証済みとコミットに記載)。DOB 実装・実飛行検証 (−67%、第 7 章参照)・1.5/1.5 の組合せは機体ごとのオプトインとして残置される: param set altitude.dob.fc 1.5 + param set altitude.vel.ti_hover 1.5。
- ・現 HEAD の params.cpp table[]: altitude.vel.ti_hover 既定 2.5 (params.cpp:742)、altitude.dob.fc 既定 0.0 (params.cpp:745)、hover.thrust_corr 既定 1.12 (params.cpp:746)。
- ・注意 (コメント残存): params.cpp:463 付近の日本語コメントに「既定 1.5Hz=有効」という差し戻し前の記述が残存しており、table[] の既定 0.0 と矛盾する。差し戻しコミットは「comments note the same-day reversal」と述べており 442-444 行付近には差し戻しの注記があるが、455-470 行のブロックには昇格時代の文が残っている。文書・コメント整備の要修正点として記録する。
- ・auto-memory 側の「既定有効化完了」という記録は差し戻し前の状態を記録したもので、現状と不一致 (メモリ側を要更新)。

2026-08-01 現在の正しい状態: 高度 DOB は「実装済み・実飛行で −67% 検証済み・既定はオフ (オプトイン)」

である。v2026.07.2 リリース (0b463766、rate.roll.td=0.002) はこの差し戻し後の既定に基づく。

3.7 ホバー推力学習

hover_thrust_ = kMassG × hover.thrust_corr (既定 1.12, params.cpp:746, pid_controller.cpp:146-148)。
kMassG = 0.037 × 9.80665 (pid_controller.hpp:407)。

learnHoverThrust() (pid_controller.cpp:968-1000) : Airborne・ALT/POS_HOLD・スロットル中立・ $|v_z| < kHoverLearnVzDead$ (0.30 m/s, pid_controller.hpp:409) のゲート下でのみ、時定数 kHoverLearnTau = 20.0 s (pid_controller.hpp:408) で速度ループの定常出力 (thrust_correction) を hover_thrust_ へ 1 次系的に畳み込む (pid_controller.cpp:998)。クランプ範囲 kHoverCorrMin=0.5~kHoverCorrMax=2.0 (pid_controller.hpp:410-411)。着陸エッジで params::set_float + params::save() により NVS 保存される (persistHoverThrust(), pid_controller.cpp:1012-1022)。hover.thrust.learn (既定 1=ON, params.cpp:747) で無効化可能。

3.8 スティック・リポジショニング機能

実装: computePositionHold() (pid_controller.cpp:1260-1352)。

- 速度源: guidance_active_ && guide_mode_==2 なら誘導由来 (Tello rc) 速度、それ以外はパイロットスティック: $v_{fwd} = -\text{setpoint.pitch} \times \text{stick_reposition_vel_}$, $v_{right} = \text{setpoint.roll} \times \text{stick_reposition_vel_}$ (pid_controller.cpp:1286-1287)。stick_reposition_vel_ は position.stick_vel (既定 0.4 m/s, params.cpp:775)。
- 再配置中 ($v_{fwd} \neq 0 \vee v_{right} \neq 0$) : 位置ループを迂回し、機体系速度指令を NED へ回転して速度ループへ直接渡す。位置ループ積分器はリセットし、保持目標を現在位置に固定し続ける (pid_controller.cpp:1300-1306)。
- 中立に戻った瞬間 (reposition_active_ → false のエッジ) : capture_pos_=true を立て、次サイクルで現在位置を新しい保持目標として捕捉 (pid_controller.cpp:1315-1323)。
- 通常保持時は通常の P 位置ループ (pid_controller.cpp:1326-1327)。

符号は STABILIZE の傾き方向と一致する (右ロール→右移動、前ピッチ→前移動、pid_controller.cpp:1266-1273 コメント)。poshold_journey.md §5 (コミット ef3f854) と整合。

3.9 自動離陸シーケンス (ALT/POS_HOLD)

実装: onTakeoff() (pid_controller.cpp:1380-1409)、onTakeoffComplete() (pid_controller.cpp:1411-1437)、compute() 内 VerticalPhase::TakeoffClimb 分岐 (pid_controller.cpp:514-567)。

- パラメータ: takeoff_climb_rate_ = 0.3 m/s (pid_controller.hpp:642)、takeoff_target_alt_ = 0.5 m (pid_controller.hpp:655)。params テーブルには見当たらず、コード内定数。
- 離陸開始 (onTakeoff()) : フェーズを TakeoffClimb に遷移、applyAltVelTiForPhase() で climb 用 ti 適用、DOB 状態リシード、鉛直ループリセット、alt_setpoint_ = takeoff_target_alt_、capture_pos_=true (POS_HOLD 用発進点捕捉)、スロットル再センターゲートを閉 (pid_controller.cpp:1388-1409)。
- 上昇中: alt_pos_.compute(alt_setpoint_, altitude, dt) の出力を ±takeoff_climb_rate_ にクランプして速度目標とし、alt_vel_.compute() で推力補正 (pid_controller.cpp:527-543)。位置ループが目標近傍で自然減速しオーバーシュートなく捕捉する設計 (コメント pid_controller.cpp:522-525)。
- 完了検出: 片側到達判定 ($\text{altitude} \geq \text{target} - k\text{TakeoffCaptureBandM}$ を $k\text{TakeoffSettleCycles}$ 継続) + タイムアウトバックストップ $k\text{TakeoffMaxCycles}$ (pid_controller.cpp:557-567、具体数値は未確認)。
- 姿勢はこの間もパイロットが完全掌握する (「鉛直のみ自動」設計、pid_controller.cpp:283-300 コメント)。

旧設計（水平強制）は 2026-06-14 の実機バグとして修正済み。

- onTakeoffComplete(): フェーズを Airborne へ、alt_vel_.ti を hover 用に切替、alt_setpoint_ は離陸目標値のまま維持（瞬時高度でなく目標値保持、行き過ぎをバイアスしない設計、pid_controller.cpp:1419-1436）。

4 状態推定 (ESKF)

実装: firmware/vehicle/components/sf_estimator_eskf/eskf_core.cpp。

4.1 状態数と内訳

15 状態: [pos(3), vel(3), att_err(3), gyro_bias(3), accel_bias(3)] (eskf_core.cpp:14)。インデックス定義: POS_X=0..BA_Z=14 (eskf_core.hpp:36-40)。姿勢は誤差状態（回転ベクトル $\delta\theta$ ）で保持し、更新後クォータニオンに $q \leftarrow q \otimes \delta q(\delta\theta)$ として注入する (eskf_core.cpp:1023-1026)。

予測モデル (predict(), eskf_core.cpp:169-) : バイアス補正済み加速度・角速度で名目状態をオイラー積分する (pos_ += vel_*dt; vel_ += (accel_ned+gravity)*dt; q_ = q_*Quat::from_rotvec(gyro*dt), eskf_core.cpp:195-199)。

4.2 観測

観測	関数	内容	出典
高度 (ToF)	updateToF()	傾き補正 height = distance · cos θ_x · cos θ_y 、傾き > tof_tilt_threshold(0.70 rad, eskf_core.hpp:84) で棄却、絶対値イノベーションゲート	eskf_core.cpp:530-555
鉛直速度 (ToF 微分)	updateToFVelocity()	$v_z = -\Delta\text{height}/dt$ 、ノイズは位置観測の 5 倍	eskf_core.cpp:557-596
水平速度 (フロー)	updateFlowRaw()	§4.3 参照	eskf_core.cpp:729-823
姿勢 (加速度計)	updateAccelAttitude()	適応 R + α - β 補償、§4.4	eskf_core.cpp:645-727
気圧高度	updateBaro()	既定 off	eskf_core.cpp:598-
地磁気	updateMag()	既定 off、 χ^2 ゲート mag_chi2_gate	eskf_core.cpp:612-642

更新はジョセフ形 ($P' = (I - KH)P(I - KH)^T + KRK^T$) とみられるが、該当コード行の個別確認は未実施であり、control_theory_overview.md:82 の記述のみ確認した（未確認）。

4.3 フロー→速度変換パイプライン

(updateFlowRaw(), eskf_core.cpp:729-823)

1. SQUAL ゲート: squal < cfg.flow_min_squal (既定 10, eskf_core.hpp:100 / params.cpp:808 eskf.gate.flow_squal) で棄却。高さゲート height < flow_min_height (0.02 m, eskf_core.hpp:89) も併用 (eskf_core.cpp:740-741)。
2. ピクセル→角速度: flow_rate = $dx \cdot \text{flow_rad_per_pixel}/dt$ (flow_rad_per_pixel=0.00222, eskf_core.hpp:87) (eskf_core.cpp:746-747)。
3. ジャイロ回転成分除去: $\text{trans}_x = \text{flow_rate}_x - \text{flow_gyro_scale} \cdot \text{gyro}_y$ (flow_gyro_scale=1.0, eskf_core.hpp:88) (eskf_core.cpp:751-752)。
4. 高度スケーリング: $v_{x, \text{body}} = \text{trans}_x \cdot \text{height}$ (eskf_core.cpp:756-757)。
5. ヨー回転で NED へ (eskf_core.cpp:761-764)。
6. スカラー更新×2 (VEL_X, VEL_Y)、イノベーションクランプ flow_innov_clamp (既定 0.3, params.cpp:807 eskf.gate.flow_clamp) (eskf_core.cpp:769-784)。

4.4 ロバスト化 5 機構

機構	実装	パラメータ既定値	出典
χ^2 外れ値ゲート	vectorUpdate3() 内 $d^2 = \text{innov}^\top S^{-1} \text{innov}$, $d^2 > \text{chi2_gate}$ で棄却	eskf.att.chi2_gate = 7.81 ($\chi^2(3, 0.95)$ と一致)	eskf_core.cpp:448-462, params.cpp:812
適応 R	$R = R_{base}^2 \cdot (1 + k(\ a\ - g)^2)$	eskf.att.k_adaptive = 10.0	eskf_core.cpp:678,685-686, params.cpp:811
active_mask P 隔離	off センサに触る状態ビットを recomputeActiveMask() で 隔離、更新時 dx[i]=0	—	eskf_core.cpp:863-, 1000-1004
ジャイロバイアス偏差クランプ	$b_g = \text{clamp}(b_{g,nominal} \pm \text{dev}, b_g)$	eskf.bias.gyro_dev_max = 0.03 rad/s	eskf_core.cpp:1047-1050, params.cpp:787
姿勢補正クランプ	dx[ATT_*] を $\pm \text{att_correction_clamp}$ で クランプ	eskf.att.corr_clamp = 0.05	eskf_core.cpp:1016-1022, params.cpp:813

4.5 α - β トラッカ (運動加速度補償)

updateFlowRaw() 内、cfg_.accel_comp_enable 時に実行される (eskf_core.cpp:799-822) :

```

vpx = flow_vel_lpf_.x + a_kin_ned_.x * kFlowDt;    // 予測 (kFlowDt=0.01, 100Hz 公称)
rx   = vx_ned - vpx;                               // 残差
flow_vel_lpf_.x = vpx + alpha * rx;                 // 速度状態更新
a_kin_ned_.x   += (beta / kFlowDt) * rx;             // 加速度状態更新 (クランプ ±amax)

```

既定値: eskf.accel_comp.enable=1, .alpha=0.2, .beta=0.02, .max=5.0 (params.cpp:816-819)。 $a_{kin, ned}$ は updateAccelAttitude() の予測比力 $h_{vec} = g_{expected} + R^\top a_{kin, ned}$ に折り込まれる (eskf_core.cpp:695-701)。生フロー速度を使用し、融合速度 vel_ は使わない設計 (コメントで明記、eskf_core.cpp:788-789、循環回避)。

4.6 アクチュエータ遅れ (≈ 110 ms) の内訳

出典: analysis/scripts/alt_dob_design/README.md §2 「鉛直アクチュエーション経路の同定」 (step1_actuation_id.py)。同定モデル:

$$y = k_T \cdot \frac{e^{-Ls}}{Ts + 1} \cdot u_{eff} + d_{ext}, \quad L \approx 62 \text{ ms (範囲 55-75ms)}, \quad T = 20 \text{ ms}, \quad k_T \approx 0.95 (\text{無次元})$$

(README.md:72-74)。

- L (純遅れ) : 離陸ランジェント 2 件 (63.6/61.2ms) の実測平均 62.4ms。独立監査 (別実装での感度掃引) で 56-70ms と安定、飛行中小信号イベントでも 62.5-77.5ms (README.md:76-79)。
- T (1 次遅れ) : tools/sysid/defaults.py の $\tau_m = 0.02$ s (コストダウン系の独立測定値) を採用 (README.md:80-81)。
- 速度ループが見る合計遅れ 0.11s (≈ 110 ms) の内訳: **アクチュエーション側 $L + T \approx 82$ ms + ESKF 速度推定側 ≈ 28 ms** (README.md:85-86)。DOB は IMU 比力を直接使うため ESKF 側の遅れを回避できると明記されている。

ファーム内定数: kDobModelDelayS=0.0624 (=62.4ms, L)、kDobModelLagS=0.02 (=20ms, T) (pid_controller.hpp:440-441)。この和 (≈ 82 ms) は「アクチュエーション側」のみであり、「 ≈ 110 ms」

は ESKF 速度推定遅れ約 28ms を加えた速度ループ全体が見る合計遅れである。この区別を本報告で明示する。

5 開発の経緯（時系列）

出典: facts_history.md §1 「開発マイルストーン時系列」。git 履歴とドキュメント・解析レポートの存在確認による。推測・補完は行っていない。

日付	コミット/出典	内容
2026-04-03	c457e8a7	ESKF_V2 実装 (active_mask 方式の P 行列分離) — vehicle_new 以前の系統
2026-04-12	2ca702d8	vehicle_new: ESKF を数学的基礎から実装
2026-05-31	0a997a95	SIL: 閉ループホバー成立、P1.2(c) ゲート G3 PASS
2026-06-01	26716dd6	SIL: ALT_HOLD 閉ループ高度保持、ヨー中も約 2cm 以内で保持
2026-06-03	cea0d8cf	SIL Plant 時間基準バグ修正 (物理 3 倍速 → 4000Hz 化 + 固定 timestep)。空中ホバー blocker 真因の解決。決定論的な空中ホバー成立の起点
2026-06-05	65c3b805, 25146b53	POS_HOLD カスケード初実装 (WIP、発散) + PMW3901 光学フロー SIL モデル追加
2026-06-06	1396339b, 9a2c9dad	運動加速度補償版 accel-attitude で POS_HOLD 全軸保持、position.vel.kp 0.3 → 0.8
2026-06-08	90093c14	χ^2 過剰棄却の根治 (accel_att_noise 0.06 → 0.8)。関連解説: 24175592
2026-06-13/14	8223e080 ほか	27 ユニット全体コードレビュー実施
2026-06-14	32fc1045, 0425484a, a4dca5cd, c63de449	ARM 起点 ALT/POS 自動離陸 (0.5m)、対称 ALT_HOLD スロットル、離陸完了時の姿勢デッドスティック、ESKF ノイズパラメータ調整 (accel_att_noise → 1.2 等)
2026-06-21	d82d1dd5	全モード姿勢トリム実装 (実飛行同定ベース)
2026-06-22	38dc38d8, 880634b0	POS_HOLD 実機発散 → 実効ゲイン再設計: position.pos.kp 0.3 → 0.4、position.vel.kp 2.0 → 3.0 (poshold_journey.md に詳細。同日 965300b4, da72e88a 常時オンボードトリム学習も追加)
2026-06-22	poshold_attID.py, poshold_loop_ design.py (6/22 16:16-16:37)	実効ゲイン同定 (実機 $K \approx 0.4g$) とカスケード再設計スクリプト作成、design_before_after.png 生成
2026-06-27	f279d68c, 39fab35, 9ce453be, 91c4e9f5	POS_HOLD 3 分飛行の長時間ふらつき解析、高度ふらつき外乱同定・D 項検討・積分強化検討
2026-07-02	2180f0dc	原稿構成計画 (MANUSCRIPT_PLAN.md) 承認・追加
2026-07-04	fc958772	オンボード・ホバー推力学習 (学習値を NVS に hover.thrust_corr として保存)
2026-07-05	32e9e7f9	vehicle_new → vehicle 昇格 (旧 vehicle は vehicle_old へ降格)。MANUSCRIPT_PLAN.md の firmware/vehicle_new/... パス記述はこの 3 日後の変更で陳腐化

日付	コミット/出典	内容
2026-07-13	3962633c, b4a333e9, 7e8cd01c, 16a7e9be	ヨートルク飽和解析スクリプト追加、旧 CSV からのヨートルク再構成、会場向け高度保持解析、位相スケジュール高度積分時間
2026-07-15	e7315999	実測モータ/プロペラ諸元（2026-07-15 計測）を物理ツール・SIL プラントへ反映
2026-07-17	0ae4deab, a9235fcb	ミキサ κ を実測値化+ヨーゲイン再スケール（NT 金沢展示でのヨー飽和対応）、ロールゲイン A/B 調整
2026-07-18	0f34a2b2, 42c984c4, 8ce52604, ffbd4e55, 1785143a	高度 DOB（外乱オブザーバ）設計→実装→初回実機 A/B →既定昇格。A/B 結果: 高度 std 55.2mm（静穏窓）vs 167.2mm ベースライン（ $\approx 67\%$ 改善）、radial rms 55.2 vs 69.3mm。同日ロール手動チューン既定化 ($k_p=1.0e-3$, $t_d=0.001$)
2026-07-22	3d1a35ed	SIL model-match gate: レートループ (b,T,L) を SIL vs 実機同定で数値比較する仕組み。注: POS_HOLD 位置ループのゲイン注入とは別物（ACRO レートループの SIL 忠実度ゲート）
2026-07-27	1626e313	MOTOR_CT 切替のログ駆動再検証。「補正不要仮説」は反証、corr 既定を 1.12 → 約 0.90 へ切替提案（未反映・意思決定待ち）
2026-07-29	50905d04, 2374fc3d, 6c997eac, 74bc7e07	飛行中熱ドリフト研究、V(T) 曲線同定の試み（形状同定不能と判明）、熱ドリフト主張の格下げ（未確定に訂正）、モータ電気特性測定手順書 (E1-E5) 作成

補足: 2026-07-13~07-29 の一連（ヨートルク飽和・MOTOR_CT・熱ドリフト）はモータ/推力モデルの精密化であり、POS_HOLD の位置・速度カスケードそのものへの変更ではない。原稿 4 章の「トルク効き 0.4~0.7 倍」の裏付けデータとして間接的に関係し得るが、MANUSCRIPT_PLAN.md 承認後にこれらを論文へ反映した形跡は確認できなかった。

5.1 POS_HOLD 「実効ゲイン欠損」の詳細

出典: poshold_journey.md §4.3 (:148-164)、control_theory_overview.md §5.4 (:255)、params.cpp:749-770 コメント。

POS_HOLD のカスケードは 2026-06-05 に初実装されたが発散していた（WIP）。2026-06-06 に運動加速度補償版 accel-attitude で全軸保持に成功し、position.vel.kp を 0.3 → 0.8 とした。しかしその後の実機飛行（2026-06-22）で SIL 上は合格していたゲインで発散が生じ、以下の実効ゲイン欠損が同定された。

- 実効「傾き→速度」ゲイン: $K \approx 0.4g$ （理想は g ）。3 手法（開ループ fit・振幅関係・校正）が一致（poshold_journey.md:150）。
- 内訳: 姿勢ループの傾き未達（指令傾きに対し実傾き ≈ 0.58 ） $\times$ フロー速度の過小読み（ $\approx 0.66g$ ）（poshold_journey.md:152-154）。 $0.58 \times 0.66 \approx 0.383 \approx 0.4$ 。
- 姿勢未達の根: 実機モータのトルク効き 0.4~0.7 倍（ACRO 発振解決時に同定済み、control_theory_overview.md:166）。
- 実機発散の症状: 周期約 9.4 秒、振幅 $\pm 0.37 \rightarrow 0.50 \rightarrow 0.62m$ と成長する緩い水平振動（不安定リミットサイクル）、 $\omega \approx 0.68 \text{ rad/s}$, $\sigma \approx +0.057/s$ （poshold_journey.md:144）。

- 理想ゲイン下の安定性: $K = g$ (理想) なら現ゲインは安定 ($\sigma = 0.124$) = **SIL 合格・実機墜落の正体** (poshold_journey.md:164)。SIL の内部プラントは理想ゲイン g を仮定しており、実機の実効ゲイン欠損 $0.4g$ を再現していなかったため、SIL 上のみ安定と判定されていたと解釈できる。
- 再設計 (poshold_loop_design.py 使用): position.vel.kp $0.8 \rightarrow 3.0$ 、position.pos.kp $1.0 \rightarrow 0.4$ (poshold_journey.md:170-173、params.cpp:771,773 現在値と一致)。ロバスト安定域: $K \in [2.8, 7]$ (傾き $\rightarrow$ 速度ゲイン, $\text{m/s}^2/\text{rad}$) , $\tau \in [50, 300]\text{ms}$ (poshold_journey.md:175)。
- 実機検証結果: 発散振動完全消失、保持精度 $\pm 6 \sim 7\text{cm}$, RMS 16mm 、定常ドリフト $31 \rightarrow 16\text{mm}$ 、有界性 (前半 RMS $0.028\text{m} \approx$ 後半 RMS 0.031m) (poshold_journey.md:177-182)。
- 限界の言明: att.kp を上げて 1.2Hz 共振のみ、att.ti を下げて 1.2Hz 共振のみ、姿勢ループのチューニングに伸びしろ無し。 $\pm 6 \sim 7\text{cm}$ が 37g 機のトルク効きでの実用限界 (poshold_journey.md:184)。

なお poshold_accel_compensation.md 末尾 (:199-210) には初回実機飛行 (2026-06-22, log 20260622T161055) 時点の暫定値として position.vel.kp $0.8 \rightarrow 2.0$, position.pos.kp $1.0 \rightarrow 0.3$ の記載があるが、これは poshold_journey.md 記載の最終確定値 (vel.kp=3.0, pos.kp=0.4) に至る前段の暫定対策であり、両文書は時系列上の異なる時点を記録している (矛盾ではなく経緯の記録、第 8 章 (6) で再掲)。params.cpp の実装値は最終確定値と一致する。

6 工夫点の整理 (論文の材料候補)

本章は事実として確認できた工夫点を列挙する。各項に出典を付す。

6.1 (a) 飛行ログからの実効ゲイン欠損の定量同定

実機の POS_HOLD 発散を、開ループ fit・振幅関係・校正の 3 手法で $K \approx 0.4g$ と一致同定した (第 5.1 節、poshold_journey.md:150)。SIL 上は理想ゲイン g を仮定していたため合格していたが、実機は $0.4g$ しか出でならず、この欠損が SIL 合格・実機発散の乖離の正体であった (poshold_journey.md:164)。

6.2 (b) 点校正でなく変動域へのロバスト再設計

単一の同定値 $K \approx 0.4g$ に対して点校正するのではなく、 $K \in [2.8, 7]$ ($\text{m/s}^2/\text{rad}$)、 $\tau \in [50, 300]\text{ms}$ という変動域を仮定してロバスト安定な再設計を行った (poshold_loop_design.py 使用、poshold_journey.md:175)。

6.3 (c) SIL (実ファームのホスト実行・決定論) と Model Identity 還流

SIL Plant 時間基準バグ修正 (2026-06-03, cea0d8cf、物理 3 倍速 $\rightarrow$ 4000Hz 化 + 固定 timestep) により決定論的な空中ホバーが成立した。POS_HOLD カスケードの初期実装 (2026-06-05) から本格的な閉ループ検証 (2026-06-01 の ALT_HOLD、2026-05-31 のホバー成立) まで、SIL が開発サイクルの中核に置かれている (第 5 章時系列)。2026-07-22 には model-match gate (レートループの SIL vs 実機同定を数値比較する仕組み、3d1a35ed) が追加されたが、これは ACRO レートループの SIL 忠実度ゲートであり、POS_HOLD 位置ループのゲイン注入とは別物である点に注意 (facts_history.md での明記)。

6.4 (d) 鉛直 DOB (IMU 比力直接使用で ESKF 遅れ 28ms を回避する設計)

速度ループが見る合計遅れ $\approx 110\text{ms}$ のうち、アクチュエーション側 $\approx 82\text{ms}$ ($L=62\text{ms}+T=20\text{ms}$) に対し、ESKF 速度推定側が $\approx 28\text{ms}$ 加わる (README.md:85-86、第 4.6 節)。DOB は IMU 比力を直接使用することでこの ESKF 側の遅れを回避する設計である。実機 A/B のログ解析では高度 std -67% (55.2 vs 167.2mm 、第 7 章) だったが、既定はオフ (オプション) である (第 3.6 節)。

追記（2026-08-01 著者判断）：上記 -67% は単一飛行のログ解析値であり、パイロット体感では効果に疑問が残った（同日既定差し戻しの背景）。このため高度 DOB は論文では成果として主張せず、今後の課題として扱う（MANUSCRIPT_PLAN.md §7 に方針決定を記録）。

6.5 (e) オンボード・ホバー推力学習（NVS 永続化）

速度ループの定常出力を時定数 20s で hover_thrust_へ畳み込み、着陸時に NVS へ永続化する（第 3.7 節、pid_controller.cpp:968-1022）。

6.6 (f) 実装細部（条件付き積分 AW・フェーズ別 ti・D-on-M 等）

条件付き積分アンチwindアップ（pid.hpp:108-133）、鉛直速度ループのフェーズ別積分時間スケジュール（applyAltVelTiForPhase()）、D-on-M（測定値微分によるキック回避、pid.hpp:20-31）、不完全微分（ $\eta = 0.125$ ）の Tustin 離散化（第 3.2 節）。

6.7 (g) INV による構造規律

アーキテクチャ不変条件 INV-1~4（第 2.7 節）により、着陸時のフィードバック並列経路バグ（2026-06-14 契機）の再発を防ぐ構造的規律を導入している。

6.8 (h) 教育目的の 4 階層アクセス

L0~L3 の 4 階層アクセス制御（第 2.3 節）により、初心者からファーム実装者まで同一コードベースを段階的に利用できる設計としている。

7 実飛行実績の数値一覧

指標定義は文書間で統一されていない（原稿計画の弱点④として記録済み、第 9 章参照）。以下は各出典の記載をそのまま転記する。

日付・出典	指標	数値
2026-06-22 poshold_journey.md	保持精度（手放しホバー）	±6~7cm, RMS 16mm
	定常ドリフト 有界性	31 → 16mm（改善前後） 前半 RMS 0.028m ≈ 後半 RMS 0.031m
2026-06-27 poshold_3min_battery_ wobble_20260627.md（178 秒の 3 分飛行）	水平位置保持	平均 drift 約 13cm, std 4.2cm, 最大 24.5cm, 卓越周波数 0.09Hz
	姿勢	roll/pitch ±7, std 1.6°（健全と判定）
	高度 std（前半, V>3.6V, ~100 秒まで）	25~40mm（良好）
	高度 std（後半, V<3.6V） 相関係数	62 → 86 → 132mm へ増大、最悪窓で p2p 約 44cm corr(電圧, 高度 std)= -0.78, corr(電圧, ホバー duty)= -1.00, corr(電圧, 水平 drift_rms)= +0.93
	電池電圧推移	地上 4.168V → 離陸直後 3.872V → 最終 3.488V（約 -2.1mV/秒の 単調な低下）
2026-07-18 高度 DOB 初回実機 A/B (0f34a2b2)	高度 std	108.7mm（全区間）/ 55.2mm（静穏窓）vs 167.2mm ベースライン
	radial rms d クランプ飽和	55.2 vs 69.3mm（同条件エアコン下） 0%（最大 78% まで）

2026-06-27 の解析結論として記載されている内容: 高度ふらつきの主因は電池消耗 ($V < 3.6V$ でのモータ推力頭打ち/非線形化) であり、水平・姿勢制御には問題がないとされている (poshold_3min_battery_wobble_20260627.md)。

8 文書・実装・記録の食い違い (生の報告)

以下は調査で発見された文書・実装・記録間の食い違いを、隠さずそのまま記録したものである。

(1) コンポーネント数の不一致

実ディレクトリ数 (`ls firmware/vehicle/components/`) は **30**: `sf_actuator`, `sf_api`, `sf_autotune`, `sf_board`, `sf_calibration`, `sf_comm`, `sf_command`, `sf_controller`, `sf_controller_pid`, `sf_core`, `sf_estimator`, `sf_estimator_complementary`, `sf_estimator_ekf`, `sf_failsafe`, `sf_hal_bmi270`, `sf_hal_bmm150`, `sf_hal_bmp280`, `sf_hal_button`, `sf_hal_buzzer`, `sf_hal_led`, `sf_hal_motor`, `sf_hal_pwm3901`, `sf_hal_power`, `sf_hal_vl53l3cx`, `sf_logger`, `sf_math`, `sf_notify`, `sf_state`, `sf_takeoff_landing`, `sf_telemetry`。

`architecture.md` 本文の日本語版は「実装コンポーネント数: **27** (HAL 10 + 責務系 14 + コア基盤 3)」(`architecture.md`:145)、英語版は「Implemented component count: **26** (HAL 10 + responsibility-mapped 14 + infra 2)」(`architecture.md`:698) と日英で数値が異なる (コア基盤 3 vs infra 2、`sf_board` の扱いが日本語版では別カウント、英語版では未反映)。英語版側には「v3 update note: 日本語 §2 は R1-R16・BSP 層・4 階層アクセスモデルで拡張されたが英語訳は未完了 (M1c で完了予定)」という明示的な注記がある (`architecture.md`:627)。

さらに実ディレクトリ数 30 に対し、文書の対応表 (`architecture.md`:125-143) には `sf_api`, `sf_autotune`, `sf_estimator_complementary` の 3 コンポーネントが**掲載されていない**。これは文書が実装の後発追加分を反映しきれていないことを示唆する (推測せず、事実として「文書記載と実ディレクトリの間に差分がある」とのみ記録する)。

(2) MANUSCRIPT_PLAN.md のパス記述の陳腐化

`MANUSCRIPT_PLAN.md` が参照する 7 箇所の `firmware/vehicle_new/...` パス記述は、2026-07-05 の `vehicle_new` → `vehicle` 昇格 (32e9e7f9、`MANUSCRIPT_PLAN.md` 承認の 3 日後) により陳腐化している。実体は同名で新パス `firmware/vehicle/` 配下に存在確認済みである。`simulator/sil/` と `logs/*.jsonl` の記載は変更なく実在する。

(3) 高度 DOB 既定値の食い違い (解決済み・真相は同日差し戻し)

第 3.6 節に記載の通り、auto-memory 記録は「既定有効化完了」(2026-07-18 の DOB 既定昇格 `ffbdae55` 時点の状態) としているが、現 HEAD の実装 (`params.cpp`) は既定 0.0 (無効) である。真相は同日約 100 分後の差し戻し (`0a3d8a39`) であり、矛盾ではなく記録更新の遅れである。加えて `params.cpp`:463 付近には昇格時代の「既定 1.5Hz=有効」というコメントが残存しており、`table[]` の既定 0.0 と字面上矛盾している (コメント整備の要修正点)。

(4) リミット値が param SSOT 外

第 3.4 節に記載の通り、位置・速度・姿勢・レート各段の出力リミット (`max_pos_tilt_`, `max_thrust_` 等) は `pid_controller.hpp` 内のコンパイル時定数であり、`params::get_float` で読み込まれない。すなわち NVS 経由のライブパラメータ変更 (param SSOT) の対象外である。

(5) 機体質量が設計文書になくコードのみ

第 2.1 節に記載の通り、機体質量 0.037kg は指定 4 文書（requirements.md / architecture.md / hardware_init.md / coding_and_education.md）のいずれにも記載がなく、pid_controller.hpp のコード内定数（コメント付き）としてのみ確認できる。

(6) poshold_accel_compensation.md と poshold_journey.md のゲイン値の違いは時系列上の別時点

第 5.1 節末尾に記載の通り、poshold_accel_compensation.md 記載の暫定ゲイン（position.vel.kp 0.8 → 2.0, position.pos.kp 1.0 → 0.3、2026-06-22 初回飛行時点）と、poshold_journey.md 記載の最終確定値（vel.kp=3.0, pos.kp=0.4）は矛盾ではなく、同一の再設計プロセス内の異なる時点（暫定対策→最終確定）を記録したものである。現在の params.cpp 実装値は最終確定値と一致する。

9 原稿執筆に向けた現状

9.1 MANUSCRIPT_PLAN.md §5 残作業チェック（facts_history.md §3 照合）

#	計画項目	状態	根拠
1	SIL への同定ゲイン注入実験(旧ゲイン発散再現・新ゲイン安定確認)	未着手	git log --all --since=2026-07-02 --grep="poshold" -i は MANUSCRIPT_PLAN 追加コミット (2180f0dc) のみ。simulator/sil/scenarios/ の pos_reposition.scn / pos_auto_takeoff.scn は既存の機能検証シナリオで、同定 $K \cdot \tau$ 注入の専用実験は見当たらない。7/22 の 3d1a35ed(model-match gate) はレートループの話で別物
2	poshold_loop_design.py によるボード線図/安定余裕図の論文品質再生成	未着手（暫定図のみ存在）	design_before_after.png (6/22 生成) は計画承認前の図。7/2以降に再生成コミットなし。bode/margin/stability 等の名称のファイルも見つからず
3	LaTeX テンプレート作成	未着手	docs/hikoki64/ には guideline.pdf, template.docx, MANUSCRIPT_PLAN.md のみ。本報告作成前は.tex ファイルなし
4	実飛行ログ logs/stampfly_udp_20260627T020137.jsonl の存在確認	存在確認済み	実在 (256591 行/55MB)
5	firmware/vehicle_new/... パスの食い違い	食い違いあり (第 8 章 (2) 参照)	MANUSCRIPT_PLAN.md が参照する 7 パス全て firmware/vehicle_new/のまま記載。実体は 2026-07-05 の昇格 (32e9e7f9) で firmware/vehicle/配下に移動済み

総括: MANUSCRIPT_PLAN.md §5 の作業計画 7 項目（うち 1 項目は任意の eskf_replay アプレーション）のうち、実質的に着手済みなのは材料の存在のみである。① SIL 注入実験、②図の論文品質再生成、③ LaTeX テンプレート、④性能指標統一比較表、⑤本文執筆はすべて確認できる範囲で未着手。締切 2026-08-07 に対し、オフライン実験部分がまるごと残っている。

9.2 解析レポート・スクリプト棚卸し

MANUSCRIPT_PLAN.md が列挙する 4 スクリプト・1 レポートはすべて実在確認済みである。ただしいずれも最終更新は 2026-06-27 までで、原稿計画承認 (2026-07-02) 以降に更新の痕跡はない (git log --since=2026-07-02 でこれらのパスへの変更コミット 0 件)。

ファイル	一行説明（存在確認済み）
analysis/scripts/poshold_attID.py	実効「傾き→速度」ゲイン K 同定（2026-06-22 17:53, 9122 bytes）
analysis/scripts/poshold_loop_design.py	位置カスケード PID 再設計・安定余裕評価（2026-06-22 16:37, 11306 bytes、Bode プロット専用関数は未確認）
analysis/scripts/poshold_alt_loop_sim.py	高度ループ再生シミュレーション（2026-06-27 02:42, 7020 bytes）
analysis/scripts/poshold_analysis.py	POS_HOLD 全体メトリクス+図生成（2026-06-22 16:16, 16723 bytes）
analysis/scripts/poshold_battery_wobble.py	3 分飛行の時間分割・電圧相関解析（2026-06-27 02:12, 4724 bytes）
analysis/reports/poshold_3min_battery_wobble_20260627.md	3 分飛行ログの高度/位置ふらつきと電池電圧の相関レポート（2026-06-27 02:12, 5850 bytes）
analysis/reports/poshold_altitude_wobble_measures_20260627.md	高度ふらつき対策の検討レポート（2026-06-27 02:29）
analysis/out/poshold/design_before_after.png	修正前後比較図（2026-06-22 16:40, 120KB、論文品質での再生成は未確認）
analysis/out/poshold/*.png（飛行ログ図 5 点）	個別フライトログの可視化図（2026-06-22～06-27）
analysis/out/poshold/metrics.json	メトリクス数値の JSON（2026-06-27 02:08, 1326 bytes）

9.3 締切までの残日数

本報告作成日 2026-08-01 時点で、締切 2026-08-07 まで **6 日**である。

10 未確認事項一覧

全材料ファイルに記載された「未確認」項目を統合して列挙する。

- コアピン留め（CPU0/CPU1 タスク割り当て）：4 指定文書中に記載なし。別の auto-memory 記録に「真因=コア 1 飽和」という記述はあるが指定文書には未記載。
- 機体質量 0.037kg の設計文書上の記載: requirements.md/architecture.md にはなし。コード（pid_controller.hpp）から確認した値であり、指定 4 文書外の出典。
- coding_and_education.md §3～5（Examples 詳細構成・チュートリアル計画・実装優先順位の本文）：見出しのみ確認、本文は未読のため詳細未確認。
- sf_api, sf_autotune, sf_estimator_complementary の 3 コンポーネントの責務定義: architecture.md の対応表に掲載なく、文書上の正式な責務記述は未確認（ディレクトリ存在のみ確認）。
- ESKF 更新のジョセフ形式: 設計文書（control_theory_overview.md:82）の記載のみ確認し、eskf_core.cpp の該当実装行は直接照合していない。
- kTakeoffCaptureBandM, kTakeoffSettleCycles の具体的数値: 未読み取り。
- altitude.vel.ti_hover の実運用値が climb 用 ti と異なるかどうか: params.cpp 既定値からは差が読めない（既定は両方 2.5）。
- ヨー軸モデル（yaw_axis_model.md）：要求範囲外のため未読了。
- poshold_loop_design.py にボード線図描画専用の関数があるか: 未確認。
- design_before_after.png の図の中身: 画像未確認、ファイル名からの推定に留まる。
- モータ電気特性・熱ドリフト系の一連（2026-07-27～07-29）が原稿 4 章「トルク効き 0.4～0.7 倍」の数値に反映される予定かどうか: 意思決定待ちの状態。